4.2 Оценка 227.27 Рейтинг

«Лаборатория Касперского»

Ловим вирусы, исследуем угрозы, спасаем мир

xoxol_89 13 сен 2018 в 11:09

Многомодульность в Android с точки зрения архитектуры. От А до Я

20 мин 61K

Блог компании «Лаборатория Касперского», Java*, Разработка мобильных приложений*, Разработка под Android*

Всем привет!

Не так давно мы с вами осознали, что мобильное приложение — это не просто тонкий клиент, а это действительно большое количество самой разной логики, которое нуждается в упорядочивании. Именно поэтому мы прониклись идеями Clean architecture, прочувствовали, что такое DI, научились использовать Dagger 2, и теперь с закрытыми глазами способны разбить любую фичу на слои.

Но мир не стоит на месте, и с решением старых проблем приходят новые. И имя этой новой проблемы — мономодульность. Обычно об этой проблеме узнаешь, когда время сборки улетает в космос. Именно так и начинаются многие доклады про переход на многомодульность (раз, два).
Но почему-то все при этом как-то забывают, что мономодульность сильно бьет не только по времени сборки, но и по вашей архитектуре. Вот ответьте на вопросы. На сколько у вас AppComponent большой? Не встречаете ли вы периодически в коде, что фича А зачем-то дергает репозиторий фичи Б, хотя вроде такого быть не должно, ну или оно должно быть как-то более верхнеуровнево? Вообще у фичи есть какой-то контракт? А как вы организовываете общение между фичами? Есть какие-то правила?
Вы чувствуете, что мы решили проблему со слоями, то есть вертикально все вроде хорошо, но вот горизонтально что-то идет не так? И просто разбиением на пакеты и контролем на ревью не решить проблему.

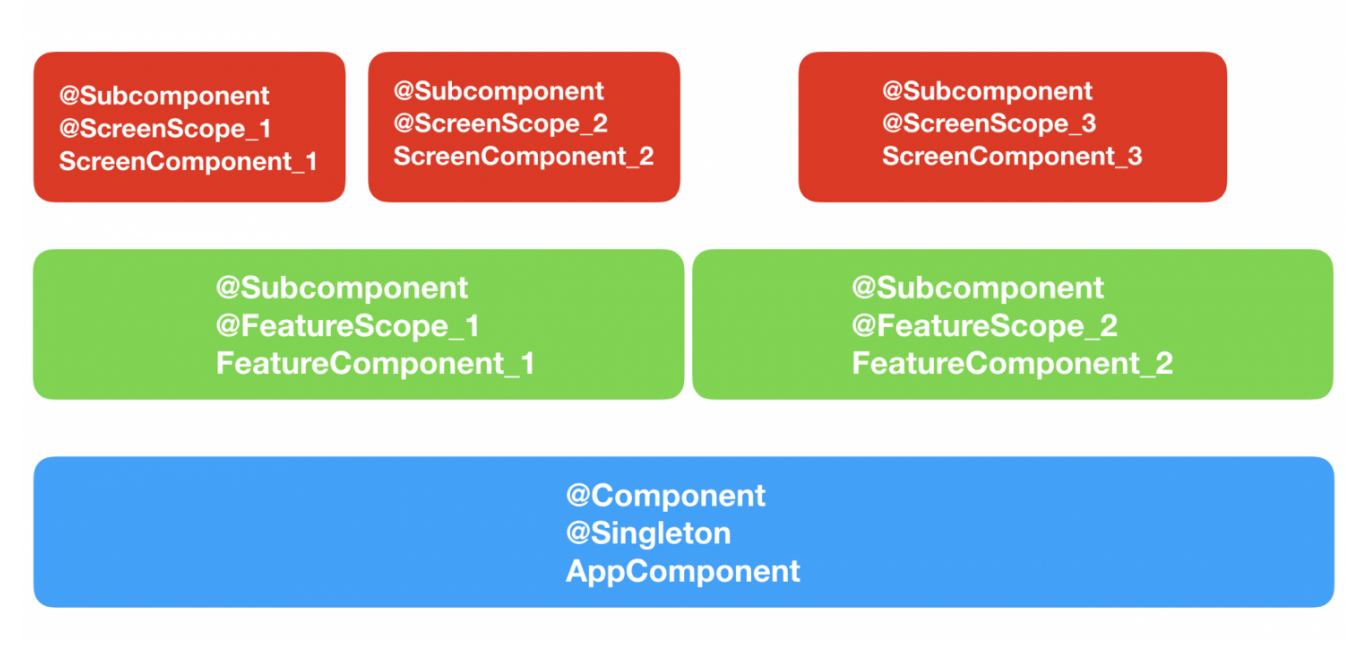
И контрольный вопрос для более опытных. Когда вы переезжали на многомодульность, не приходилось ли вам перелопачивать половину приложения, вечно перетаскивать код с одного модуля в другой и жить с несобирающимся проектом приличный отрезок времени?

В своей статье я хочу вам рассказать, как дошел до многомодульности именно с архитектурной точки зрения. Какие проблемы меня беспокоили, и как я их старался поэтапно решать. А в конце вас ждет алгоритм перехода с мономодульности на многомодульность без слез и боли.

Отвечая на первый вопрос, на сколько у меня большой AppComponent, я могу признаться — большой, реально большой. И это меня постоянно терзало. Как так вышло? Прежде всего это из-за такой организации DI. Именно с DI мы и начнем.

Как я делал DI раньше

Думаю, у многих в голове сформировалась примерно такая схема зависимостей компонентов и соответствующих скоупов:


@Subcomponent
@ScreenScope_1
ScreenComponent_1
@Subcomponent
@ScreenScope_2
ScreenComponent_2
@Subcomponent
@ScreenScope_3
ScreenComponent_3
@Subcomponent
@FeatureScope_1
FeatureComponent_1
@Subcomponent
@FeatureScope_2
FeatureComponent_2
@Component
@Singleton
AppComponent

Что мы тут имеем

AppComponent, который вбирал в себя абсолютно все зависимости со скоупом *Singleton*. Думаю, этот компонент есть практически у всех.

FeatureComponents. Каждая фича была со своим скоупом и являлась сабкомпонентом *AppComponent* или старшей фичи.
Давайте немного остановимся на фичах. Прежде всего, что такое фича? Постараюсь своими словами. **Фича** — это логически законченный, максимально независимый модуль программы, решающий конкретную пользовательскую проблему, с четко обозначенными внешними зависимостями, и который относительно легко использовать снова в другой программе. Фичи могут быть большими и маленькими. Фичи могут содержать другие фичи. А могут также использовать или запускать другие фичи через четко обозначенные внешние зависимости. Если взять наше приложение (Kaspersky Internet Security for Android), то фичами можно считать Антивирус, Антивор, и т.д.

ScreenComponents. Компонент для конкретного экрана, также со своим скоупом и также являющийся сабкомпонентом от соответствующего фиче-компонента.

Теперь список из «почему так»

Почему сабкомпоненты?
В component dependencies мне не нравилось прежде всего то, что компонент может зависеть сразу от нескольких компонентов, что, как мне казалось, могло в конечном счете привести к хаосу компонентов и их зависимостей. Когда же у тебя строгая зависимость вида «один ко многим» (компонент и его сабкомпоненты), то так безопаснее и очевиднее. Кроме того сабкомпоненту по умолчанию доступны все зависимости родителя, что также вроде удобнее.

Почему на каждую фичу свой скоуп?
Потому что тогда исходил из соображений, что каждая фича — это какой-то свой ЖЦ, который не такой, как у других, поэтому логично создать свой скоуп. Есть еще один момент за много скупов, о котором упомяну ниже.

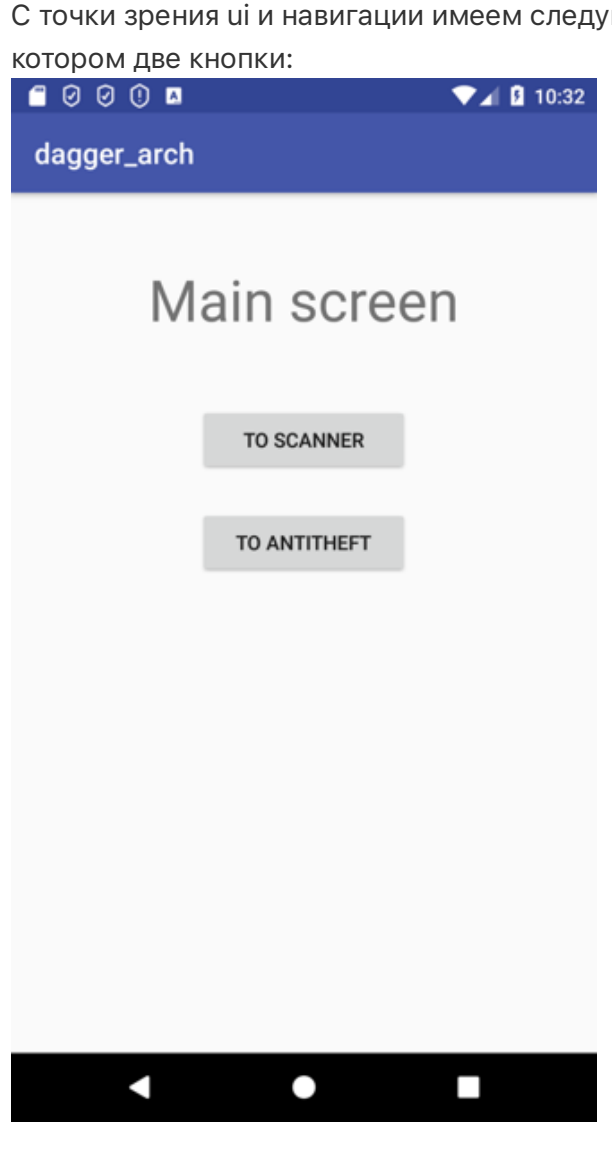
Так как говорим мы про Dagger 2 в разрезе Clean, то упомяну и про момент, как доставлялись зависимости. В Презентеры, Интеракторы, Репозитории и прочие вспомогательные классы зависимости поставлялись через конструктор. В тестах мы тогда через конструктор подставляем стабы или моки и спокойно тестируем наш класс.
Замыкание графа зависимостей происходит обычно в активити, фрагменты, иногда ресиверы и сервисы, в общем, в корневые места, с которых андроид может что-то стартовать. Классическая ситуация, когда для фичи создается активити, в активити стартует и живет компонент фичи, а в самой фиче есть три экрана, которые имплементированы в три фрагмента.

Итак, вроде все логично. Но как всегда жизнь вносит свои коррективы.

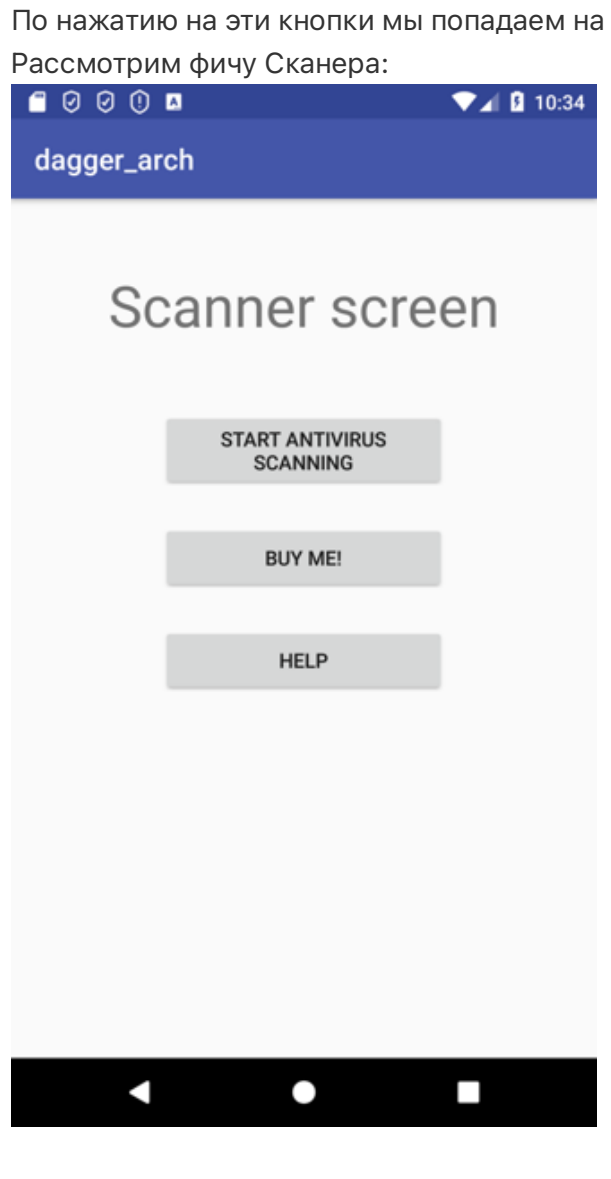
Жизненные проблемы

Задача-пример

Давайте рассмотрим простой пример из нашего приложения. У нас есть фича Сканирования (Scanner) и фича Антивора (Antitheft). В обеих фичах есть заветная кнопка «Купить». Причем «Купить» — это не просто послать запрос, а еще очень много всякой разной логики, связанной с процессом покупки. Это чисто бизнес-логика с некоторыми диалогами для непосредственной покупки. То есть налицо вполне себе отдельная фича — Покупка (Purchase). Таким образом, в двух фичах нам нужно задействовать третью фичу.
С точки зрения ui и навигации имеем следующую картину. Запускается главный экран, на котором две кнопки:


10:32
dagger_arch
Main screen
TO SCANNER
TO ANTITHEFT

По нажатию на эти кнопки мы попадаем на фичу Сканера или Антивора.
Рассмотрим фичу Сканера:


10:34
dagger_arch
Scanner screen
START ANTIVIRUS SCANNING
BUY ME!
HELP

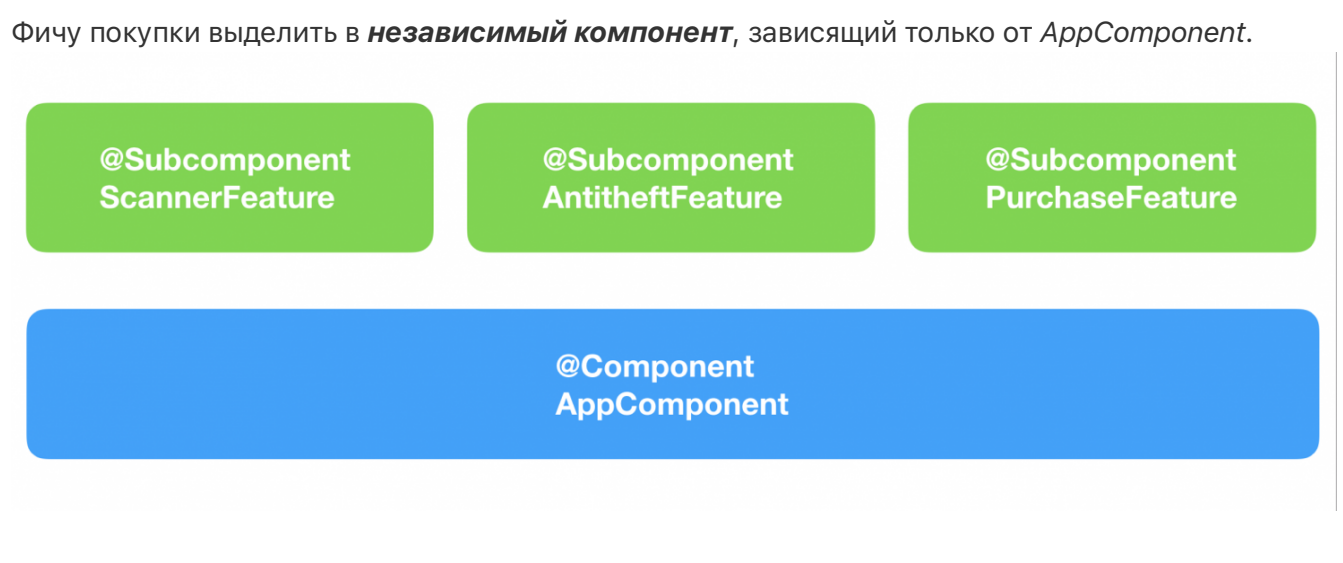
По нажатию на «Start antivirus scanning» выполняется какая-то работа по сканированию, по нажатию на «Buy me» мы как раз хотим купить, то есть дергаем фичу Покупки, ну а по «Help» — попадаем на простой экран с хэлпом.
Фича Антивора выглядит практически аналогично.

Потенциальные решения

Как нам реализовать данный пример с точки зрения DI? Есть несколько вариантов.

Первый вариант

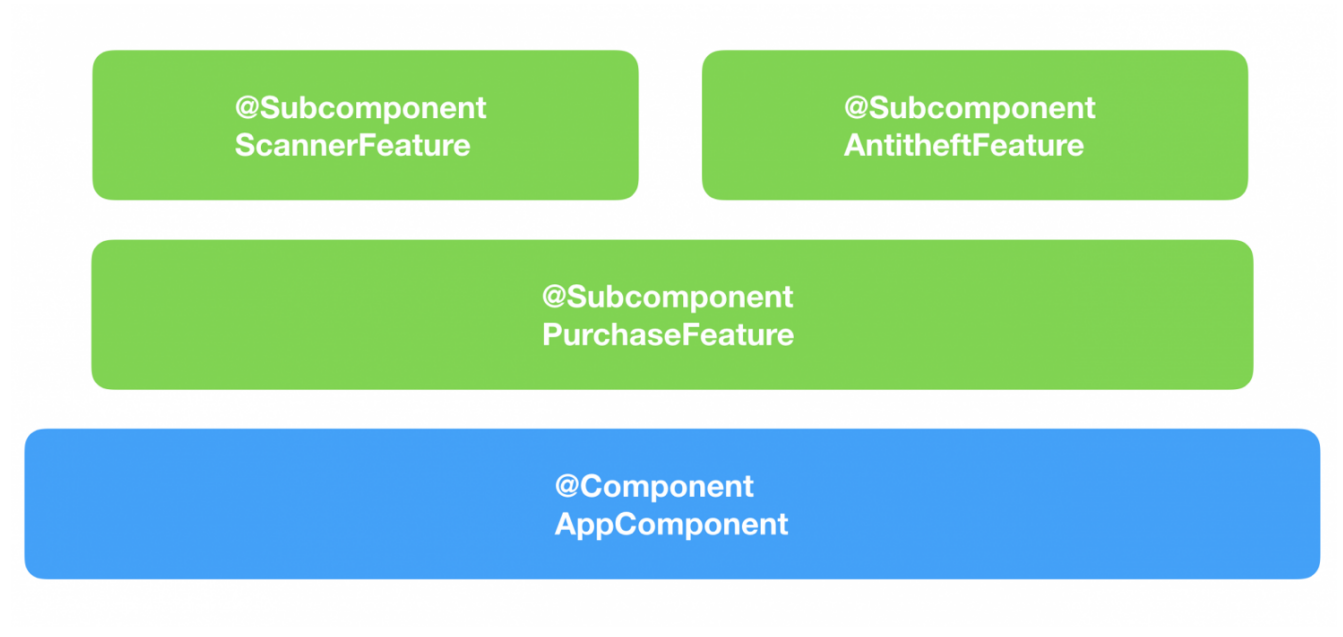
Фичу покупки выделить в ***независимый компонент***, зависящий только от *AppComponent*.


@Subcomponent
ScannerFeature
@Subcomponent
AntitheftFeature
@Subcomponent
PurchaseFeature
@Component
AppComponent

Но тогда мы сталкиваемся с проблемой: как в один класс заинжектить зависимости сразу от двух разных графов (компонентов)? Только через грязные костыли, что, конечно, такое себе.

Второй вариант

Фичу покупки выделяем в сабкомпонент, зависящий от AppComponent. А компоненты Сканера и Антивора сделать сабкомпонентами уже от компонента Покупки.

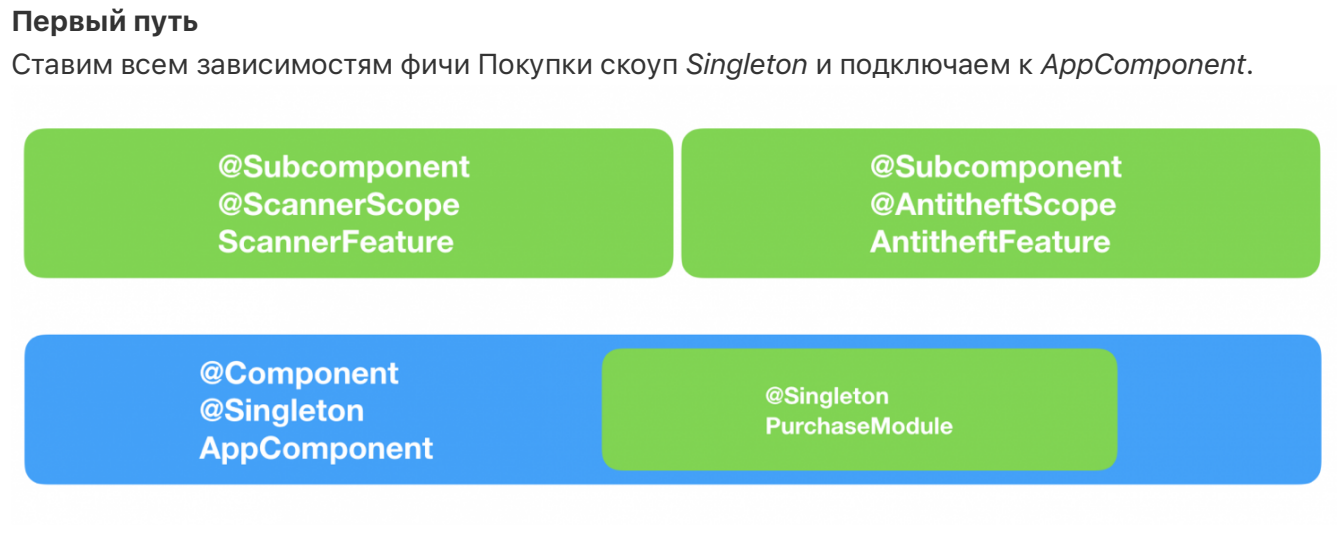

@Subcomponent
ScannerFeature
@Subcomponent
AntitheftFeature
@Subcomponent
PurchaseFeature
@Component
AppComponent

Но, как вы понимаете, подобных ситуаций может быть довольно много в приложениях. А это означает, что глубина зависимостей компонентов может быть воистину огромной и сложной. И подобный граф будет скорее запутывать, нежели делать ваше приложение более стройным и понятным.

Третий вариант

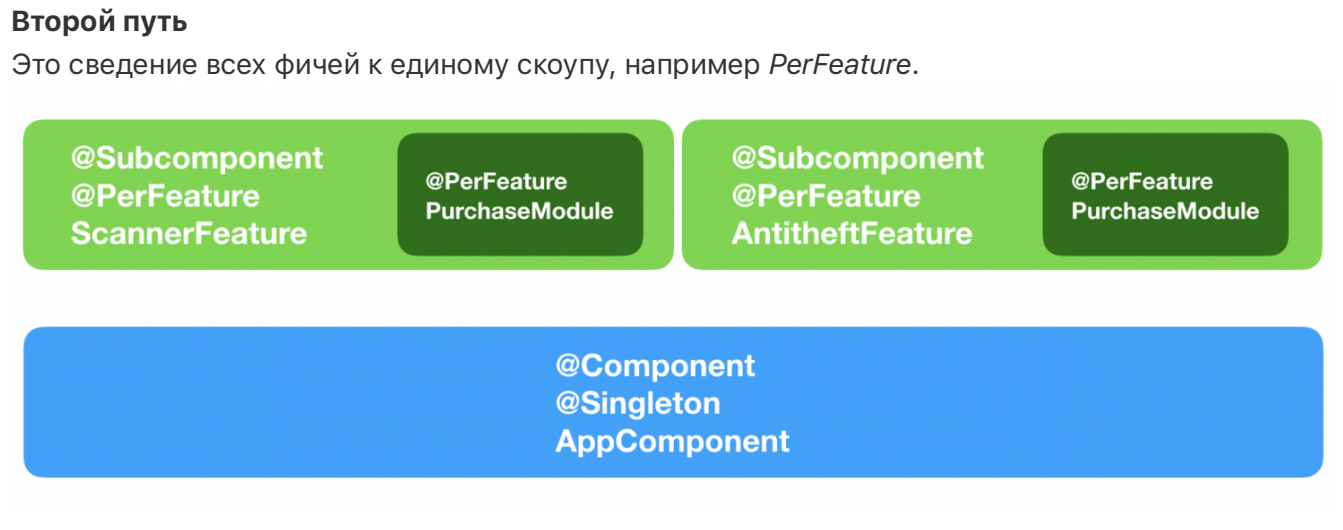
Фичу покупки выделяем **не в отдельный компонент, а в отдельный даггеровский модуль**.
Далее возможны два пути.

Первый путь
Ставим всем зависимостям фичи Покупки скоуп *Singleton* и подключаем к *AppComponent*.


@Subcomponent
@ScannerScope
ScannerFeature
@Subcomponent
@AntitheftScope
AntitheftFeature
@Component
@Singleton
AppComponent
@Singleton
PurchaseModule

Вариант популярный, но влечет к раздуванию *AppComponent*. В итоге он раздувается в размерах, содержит в себе все классы приложения, и вся суть использования Dagger сводится только к более удобной доставке зависимостей в классы — через поля или конструктор, а не через синглтоны. В принципе, это же и есть DI, но мы упускаем архитектурные моменты, и получается, что все знают обо всех.
Вообще в начале пути, если не знаешь, куда отнести какой-то класс, к какой фиче, то проще его сделать глобальным. Такое довольно распространено, когда работаешь с легаси и пытаешься привнести туда хоть какую-то архитектуру, плюс ты не знаешь еще хорошо весь код. И там действительно глаза разбегаются, и данные действия оправданны. Ошибка в том, что когда все более-менее вырисовывается, никто не хочет браться за этот *AppComponent*.

Второй путь
Это сведение всех фичей к единому скоупу, например *PerFeature*.


@Subcomponent
@PerFeature
ScannerFeature
@PerFeature
PurchaseModule
@Subcomponent
@PerFeature
AntitheftFeature
@PerFeature
PurchaseModule
@Component
@Singleton
AppComponent

Тогда мы сможем даггеровский модуль Покупки подключать к необходимым компонентам легко и просто.
Вроде удобно. Но архитектурно получается не изолированно. Фичи Сканера и Антивора знают абсолютно все о фиче Покупки, все ее потроха. По неосторожности что-то может быть задействовано. То есть у фичи Покупки отсутствует четкий API, граница между фичами размытая, отсутствует четкий контракт. Это плохо. Ну и в многомодульность градловую будет тяжело потом.

Архитектурная боль

Честно признаюсь, долгое время я использовал **третий вариант.первый путь**. Это была вынужденная мера, когда мы наше легаси начали постепенно переводить на нормальные рельсы. Но, как я уже упоминал, при данном подходе ваши фичи начинают немного смешиваться. Каждый может знать о каждом, о деталях реализации и вот этом всем. И раздувание *AppComponent* явно говорило о том, что нужно что-то делать.
Кстати говоря, с разгрузкой именно *AppComponent* хорошо помог бы **третий вариант.второй путь**. Но вот знания об имплементациях и смешение фичей никуда не денутся. Ну и понятное дело, переиспользование фичей между приложениями было бы весьма непростым делом.

Промежуточные выводы

Итак, что же в итоге нам хочется? Какие проблемы мы хотим решить? Давайте прямо по пунктам, начиная от DI и переходя к архитектуре:

- Удобный механизм DI, позволяющий использовать фичи в рамках других фичей (в нашем примере мы хотим в рамках Сканера и Антивора использовать фичу Покупок), без костылизации и боли.
- Тончайший AppComponent.
- Фичи не должны знать об имплементациях других фичей.
- Фичи не должны быть доступны по умолчанию кому угодно, хочется иметь какой-то строгий механизм контроля.
- Возможно отдать фичу в другое приложение с минимальным количеством телодвижений.
- Логичный переход на многомодульность и лучшие практики по этому переходу.

Я специально сказал про многомодульность только в самом конце. Мы дойдем до нее, не будем забегать вперед.

"Життя по-новому"

Теперь мы с вами постараемся поэтапно реализовывать обозначенные выше хотелки.
Поехали!

Улучшения DI

Начнем все с того же DI.

Отказ от большого количества скоупов

Как я писал выше, раньше мой подход был таким: на каждую фичу свой скоуп. На самом деле каких-либо особых профитов от этого нет. Только получаете большое количество скоупов и некоторое количество головной боли.
Вполне достаточно такой цепочки: *Singleton* — *PerFeature* — *PerScreen*.

Отказ от Subcomponents в пользу Component dependencies

Уже более интересный момент. С *Subcomponents* вы имеете вроде более строгую иерархию, но при этом у вас полностью связаны руки и нет возможности хоть как-то маневрировать. Кроме того, *AppComponent* знает обо всех фичах, и еще вы получаете огромный сгенерированный класс *DaggerAppComponent*.
С *Component dependencies* вы получаете одно суперкрутое преимущество. В зависимостях компонентов вы можете указывать **не компоненты, а чистые интерфейсы** (спасибо Денису и Володе). Благодаря этому вы можете подставлять какие угодно имплементации интерфейса, Даггер все съест. Даже если этой имплементацией будет компонент с таким же скоупом:

```
@Component(
    dependencies = FeatureDependencies.class,
    modules = FeatureModule.class
)
@PerFeature
public abstract class FeatureComponent {
    // ...
}

public interface FeatureDependencies {
    SomeDependency someDependency();
}

@Component(
    modules = AnotherFeatureModule.class
)
@PerFeature
public abstract class AnotherFeatureComponent implements FeatureDependencies {
    // ...
}
```

От улучшений DI к улучшению архитектуры

Давайте еще раз повторим определение фичи. **Фича** — это логически законченный, максимально независимый модуль программы, решающий конкретную пользовательскую проблему, с четко обозначенными внешними зависимостями, и который относительно легко переиспользовать в другой программе. Одно из ключевых выражений в определении фичи — это «с четко обозначенными внешними зависимостями». Поэтому давайте все, что мы хотим от внешнего мира для фичи, будем описывать в специальном интерфейсе.
Вот, допустим, интерфейс внешних зависимостей фичи Покупки:

```
public interface PurchaseFeatureDependencies {
    HttpClientApi httpClient();
}
```

Или интерфейс внешних зависимостей фичи Сканера:

```
public interface ScannerFeatureDependencies {

    DbClientApi dbClient();
    HttpClientApi httpClient();
    SomeUtils someUtils();
    // Фиче Сканера нужна возможность осуществлять покупки
    PurchaseInteractor purchaseInteractor();

}
```

Как уже было сказано в разделе о DI, зависимости могут имплементироваться кем угодно и как угодно, это же чистые интерфейсы, и наши фичи освобождены от этих лишних знаний.

Другая важная составляющая «чистой» фичи — это наличие четкого апи, по которому внешний мир может обращаться к фиче.
Вот апи фичи Покупки:

```
public interface PurchaseFeatureApi {
    PurchaseInteractor purchaseInteractor();
}
```

То есть внешний мир может получить *PurchaseInteractor* и через него пробовать осуществить покупку. Собственно выше мы и увидели, что Сканеру нужен *PurchaseInteractor* для осуществления покупки.

А вот апи фичи Сканера:

```
public interface ScannerFeatureApi {
    ScannerStarter scannerStarter();
}
```

И сразу привожу интерфейс и имплементацию *ScannerStarter*:

```
public interface ScannerStarter {
    void start(Context context);
}

@PerFeature
public class ScannerStarterImpl implements ScannerStarter {

    @Inject
    public ScannerStarterImpl() {
    }

    @Override
    public void start(Context context) {
        Class<?> cls = ScannerActivity.class;
        Intent intent = new Intent(context, cls);
        intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
        context.startActivity(intent);
    }

}
```

Тут интереснее. Дело в том, что Сканер и Антивор довольно замкнутые и изолированные фичи. В моем примере эти фичи запускаются на отдельных Активити, со своей навигацией и т. д. То есть нам здесь достаточно просто стартовать Активити. Умирает Активити — умирает и фича. Вы же можете работать по принципу "Single Activity", и тогда через апи фичи передавать, допустим, FragmentManager и какой-нибудь колбек, через который фича сообщает, что она завершилась. Вариаций много.
Можно также сказать, что такие фичи, как Сканер и Антивор, мы в праве рассматривать как независимые приложения. В отличие от фичи Покупки, которая является фичей-дополнением к чему-либо и сама по себе как-то не особо может существовать. Да, она независимая, но является логичным дополнением к другим фичам.

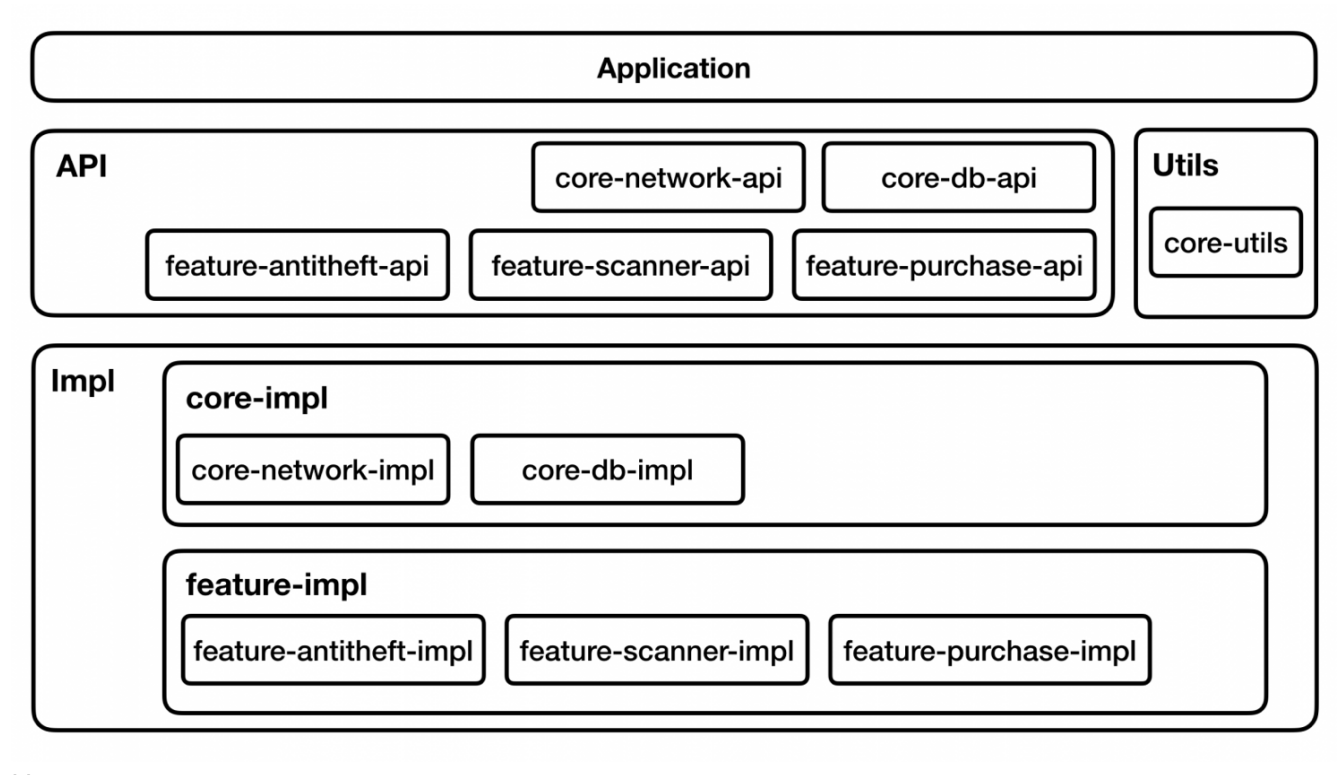
Как вы догадываетесь, должна существовать какая-то точка, которая связывает апи фичи, ее имплементацию и необходимые фиче зависимости. Этой точкой является даггеровский компонент.

- Пример компонента фичи Сканера:

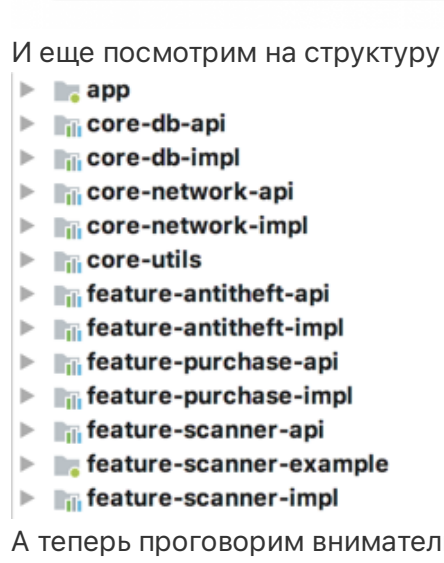
Думаю, ничего нового для вас.

Переход к многомодульности

Итак, нам с вами удалось четко обозначить границы фичи через апи ее зависимостей и внешнее апи. Также мы разобрались, как это все провернуть в Даггере. А теперь мы подходим к следующему логичному и интересному шагу — разбиению на модули.
Сразу открывайте тестовый пример — дело пойдет проще.
Давайте посмотрим на картину в общем:


Application
API
core-network-api
core-db-api
feature-antitheft-api
feature-scanner-api
feature-purchase-api
Utils
core-utils
Impl
core-impl
core-network-impl
core-db-impl
feature-impl
feature-antitheft-impl
feature-scanner-impl
feature-purchase-impl

И еще посмотрим на структуру пакетов примера:


app
core-db-api
core-db-impl
core-network-api
core-network-impl
core-utils
feature-antitheft-api
feature-antitheft-impl
feature-purchase-api
feature-purchase-impl
feature-scanner-api
feature-scanner-example
feature-scanner-impl

А теперь проговорим внимательно каждый пункт.

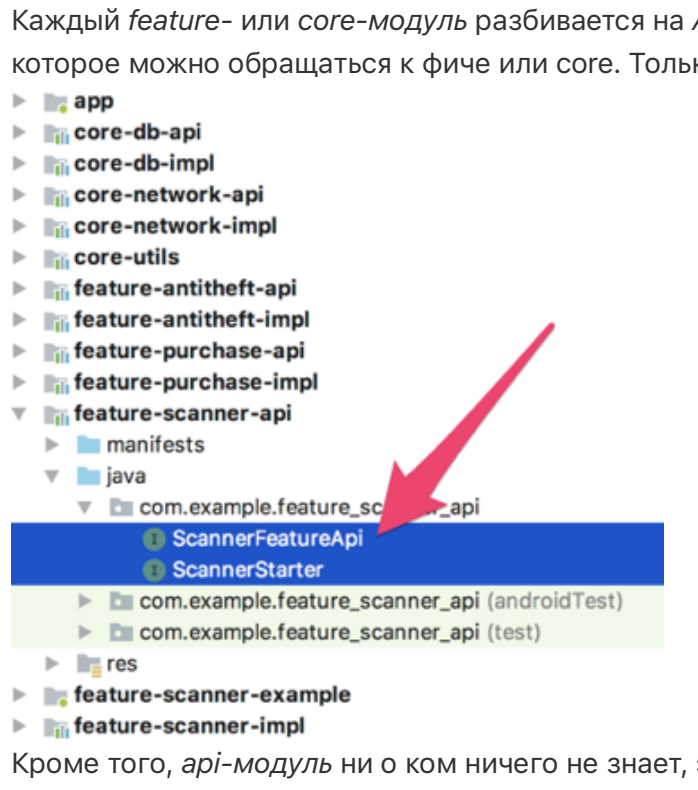
В первую очередь мы видим четыре больших блока: **Application**, **API**, **Impl** и **Utils**. В *API*, *Impl* и *Utils* вы можете заметить, что все модули начинаются или на **core-**, или на **feature-**. Давайте для начала поговорим про них.

Разделение на core и feature

Все модули я разделяю на две категории: *core-* и *feature-*.
В *feature-*, как вы могли догадаться, наши фичи. В **core-** находятся такие вещи, как утилиты, работа с сетью, бд и т. д. Но там нет каких-то интерфейсов фич. И *core* — не монолит. Я за разбиение *core-модуля* на логические кусочки и против загрузки его еще какими-то интерфейсами фич.
В названии модуля первым пишем *core* или *feature*. Далее в названии модуля идет логическое название (*scanner*, *network* и т.д.).

Теперь про четыре больших блока: Application, API, Impl и Utils

API
Каждый *feature-* или *core-модуль* разбивается на *API* и *Impl*. В *API* находится внешнее апи, через которое можно обращаться к фиче или core. Только это, и ничего более:


app
core-db-api
core-db-impl
core-network-api
core-network-impl
core-utils
feature-antitheft-api
feature-antitheft-impl
feature-purchase-api
feature-purchase-impl
feature-scanner-api
manifests
java
com.example.feature_scanner_api
ScannerFeatureApi
ScannerStarter
com.example.feature_scanner_api (androidTest)
com.example.feature_scanner_api (test)
res
feature-scanner-example
feature-scanner-impl

Кроме того, *api-модуль* ни о ком ничего не знает, это абсолютно изолированный модуль.

Utils
Единственным исключением из правила выше можно считать какие-то ну совсем утилитные вещи, которые разбивать на апи и имплементацию бессмысленно.

Impl
Тут у нас есть подразбиение на *core-impl* и *feature-impl*.
Модули в *core-impl* также абсолютно независимы. Единственная их зависимость — это *api-модуль*. Для примера взглянем на *build.gradle* модуля *core-db-impl*:

```
// bla-bla-bla
dependencies {
    implementation project(':core-db-api')
    // bla-bla-bla
}
```

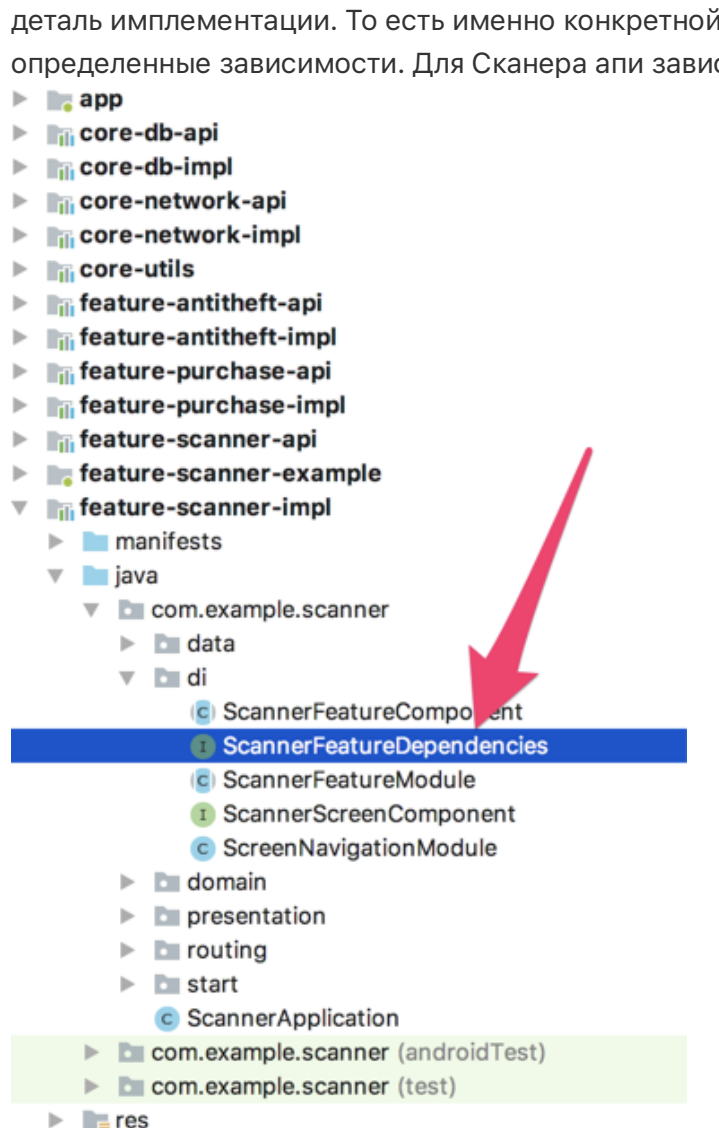
Теперь про *feature-impl*. Тут уже находится львиная доля логики приложения. Модули группы *feature-impl* могут знать про модули группы *API* или *Utils*, но точно ничего не знают о других модулях группы *Impl*.
Как мы помним, все внешние зависимости фичи аккумулируются в апи внешних зависимостей. Например, для фичи Скана это апи выглядит следующим образом:

```
public interface ScannerFeatureDependencies {

    // core-db-api
    DbClientApi dbClient();
    // core-network-api
    HttpClientApi httpClient();
    // core-utils
    SomeUtils someUtils();
    // feature-purchase-api
    PurchaseInteractor purchaseInteractor();

}
```

Соответственно, *build.gradle feature-scanner-impl* будет таким:

```
// bla-bla-bla
dependencies {
    implementation project(':core-utils')
    implementation project(':core-network-api')
    implementation project(':core-db-api')
    implementation project(':feature-purchase-api')
    implementation project(':feature-scanner-api')
    // bla-bla-bla
}
```

Вы можете спросить, а почему апи внешних зависимостей не в апи-модуле? Дело в том, что это деталь имплементации. То есть именно конкретной имплементации нужны какие-то определенные зависимости. Для Сканера апи зависимостей находится вот здесь:


app
core-db-api
core-db-impl
core-network-api
core-network-impl
core-utils
feature-antitheft-api
feature-antitheft-impl
feature-purchase-api
feature-purchase-impl
feature-scanner-api
feature-scanner-example
feature-scanner-impl
manifests
java
com.example.scanner
data
di
ScannerFeatureComponent
ScannerFeatureDependencies
ScannerFeatureModule
ScannerScreenComponent
ScreenNavigationModule
domain
presentation
routing
start
ScannerApplication
com.example.scanner (androidTest)
com.example.scanner (test)
res

Небольшое архитектурное отступление
Давайте переварим все вышесказанное и уясним для себя некоторые архитектурные моменты, касающиеся *feature-...-impl-модулей* и их зависимостей от других модулей.
Я встречал два наиболее популярных паттерна выставления зависимостей для модуля:

- Модуль может знать о ком угодно. Никаких правил нет. Тут даже комментировать нечего.
- Модули знают только о *core-модуле*. А в *core-модуле* сосредоточены все интерфейсы всех фич. Такой подход мне не очень импонирует, так как есть риск превратить *core* в очередную помойку. Кроме того, если мы захотим перенести наш модуль в другое приложение, то должны будем скопипастить эти интерфейсы в другое приложение, и также его поместить в *core*. Сам по себе тупой копипаст именно интерфейсов не очень привлекателен и реюзабелен в дальнейшем, когда интерфейсы могут обновиться.

В нашем же примере я выступаю за знание модулями апи и только апи (ну и utils-группы). Фичи абсолютно ничего не знают об имплементациях.

Но получается, что фичи могут знать о других фичах (через api, конечно же) и их запускать. Не получится ли в итоге каша?
Справедливое замечание. Тут тяжело выработать какие-то суперчеткие правила. Во всем должна быть мера. Мы немного уже касались этого вопроса выше, разделяя фичи на независимые (Сканер и Антивор) — вполне независимые и обособленные, и фичи «в контексте», то есть запускаемые всегда в рамках чего-то (Покупка) и обычно подразумевающие под собой бизнес-логику без ui. Именно поэтому Сканер и Антивор знают о Покупках.
Другой пример. Представим, что в Антиворе есть такая штука, как wipe data, то есть очищение абсолютно всех данных с телефона. Там много бизнес-логики, ui, оно вполне обособлено. Поэтому логично выделить wipe data в отдельную фичу. И тут развилка. Если wipe data всегда запускается только с Антивора и всегда присутствует в Антиворе, то логично, чтобы Антивор знал бы о wipe data и самостоятельно запускал ее. А аккумулирующий модуль, app, знал бы тогда только об Антиворе. Но если wipe data может запускаться еще где-то или не всегда присутствует в Антиворе (то есть в разных приложениях может быть по-разному), то логично, чтобы Антивор не знал об этой фиче и просто говорил чему-то внешнему (через Router, через какой-то колбек, это неважно), что пользователь нажал такую-то кнопку, а что под ней запускать — это уже дело потребителя фичи Антивора (конкретное приложение, конкретный app).

Также еще есть интересный вопрос о переносе фичи в другое приложение. Если мы, допустим, захотим перенести Сканер в другое приложение, то мы должны также перенести помимо модулей *:feature-scanner-api* и *:feature-scanner-impl* и модули, от которых Сканер зависит (*:core-utils, :core-network-api, :core-db-api, :feature-purchase-api*).
Да, но! Во-первых, все ваши api-модули абсолютно независимы, и там только интерфейсы и модели данных. Никакой логики. И эти модули четко разделены логически, а *:core-utils* — обычно общий модуль для всех приложений.
Во-вторых, вы можете апи-модули собирать в виде aar и поставлять через мавен в другое приложение, а можете подключать в виде гитового саб-модуля. Но у вас будет версионирование, будет контроль, будет цельность.
Таким образом, переиспользование модуля (точнее модуля-имплементации) в другом приложении выглядит гораздо проще, понятнее и безопаснее.

Application

Вроде у нас вырисовывается стройная и понятная картина с фичами, модулями, их зависимостями и вот этим всем. Теперь мы подходим к кульминации — это соединение апи и их имплементаций, подставление всем необходимых зависимостей и т. д., но уже с точки зрения гредловых модулей. Точкой соединения служит обычно сам *app*.
Кстати, в нашем примере такой точкой еще является *feature-scanner-example*. Вышеописанный подход позволяет вам запускать каждую свою фичу как отдельное приложение, что сильно экономит время сборки во время активной разработки. Красота!

Рассмотрим для начала, как все через *app* происходит на примере уже полюбившегося Сканера.

▸ Быстро вспомним фичу:

На основе этого мы можем создать даггеровский компонент, имплементирующий api внешних зависимостей:

```
@Component(dependencies = {
        CoreUtilsApi.class,
        CoreNetworkApi.class,
        CoreDbApi.class,
        PurchaseFeatureApi.class
    })
@PerFeature
interface ScannerFeatureDependenciesComponent extends ScannerFeatureDependencies { }
```

Данный интерфейс я разместил в *ScannerFeatureComponent* для удобства:

```
@Component(modules = {
    ScannerFeatureModule.class,
    ScreenNavigationModule.class
}, dependencies = ScannerFeatureDependencies.class)
@PerFeature
public abstract class ScannerFeatureComponent implements ScannerFeatureApi {

    // bla-bla-bla

    @Component(dependencies = {
        CoreUtilsApi.class,
        CoreNetworkApi.class,
        CoreDbApi.class,
        PurchaseFeatureApi.class
    })
    @PerFeature
    interface ScannerFeatureDependenciesComponent extends ScannerFeatureDependencies

}
```

Теперь App. App знает обо всех необходимых ему модулях (*core-, feature-, api, impl*):

```
// bla-bla-bla
dependencies {
    implementation project(':core-utils')
    implementation project(':core-db-api')
    implementation project(':core-db-impl')
    implementation project(':core-network-api')
    implementation project(':core-network-impl')
    implementation project(':feature-scanner-api')
    implementation project(':feature-scanner-impl')
    implementation project(':feature-antitheft-api')
    implementation project(':feature-antitheft-impl')
    implementation project(':feature-purchase-api')
    implementation project(':feature-purchase-impl')
    // bla-bla-bla
}
```

Далее создаем вспомогательный класс. Например, *FeatureProxyInjector*. Он будет помогать правильно инициализировать все компоненты, и именно через этот класс мы будем обращаться к апи фичей. Давайте посмотрим, как у нас инициализируется компонент фичи Сканера:

```
public class FeatureProxyInjector {

    // another...

    public static ScannerFeatureApi getFeatureScanner() {
        return ScannerFeatureComponent.initAndGet(
            DaggerScannerFeatureComponent_ScannerFeatureDependenciesComponent.builder
                .coreDbApi(CoreDbComponent.get())
                .coreNetworkApi(CoreNetworkComponent.get())
                .coreUtilsApi(CoreUtilsComponent.get())
                .purchaseFeatureApi(featurePurchaseGet())
                .build()
        );
    }

}
```

Наружу мы отдаем интерфейс фичи (*ScannerFeatureApi*), а внутри как раз инициализируем весь граф зависимостей имплементации (через метод *ScannerFeatureComponent.initAndGet(...)*).
DaggerPurchaseComponent_PurchaseFeatureDependenciesComponent — это сгенерированная Даггером имплементация *PurchaseFeatureDependenciesComponent*, про который мы говорили выше, где в билдер подставляем имплементации апи-модулей.
Вот и вся магия. Посмотрите еще раз пример.

Кстати, об *example*. В *example* мы также должны удовлетворить все внешние зависимости *:feature-scanner-impl*. Но так как это пример, то мы можем подставить классы-пустышки.
Как это будет выглядеть:

```
// создаем вот такую реализацию ScannerFeatureDependencies
public class ScannerFeatureDependenciesFake implements ScannerFeatureDependencies {

    @Override
    public DbClientApi dbClient() {
        return new DbClientFake();
    }

    @Override
    public HttpClientApi httpClient() {
        return new HttpClientFake();
    }

    @Override
    public SomeUtils someUtils() {
        return CoreUtilsComponent.get().someUtils();
    }

    @Override
    public PurchaseInteractor purchaseInteractor() {
        return new PurchaseInteractorFake();
    }

}

// и где-нибудь в Application-файле инициализируем граф
public class ScannerExampleApplication extends Application {

    @Override
    public void onCreate() {
        super.onCreate();
        ScannerFeatureComponent.initAndGet(
            // да, Даггер отлично съедает это =)
            new ScannerFeatureDependenciesFake()
        );
    }

}
```

А саму фичу Сканера в *example* запускаем через манифест, чтобы не городить дополнительных пустых активити:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.scanner_example">

    <application
        android:name=".ScannerExampleApplication"
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <!--Вот она активити Сканера-->
        <activity android:name="com.example.scanner.presentation.view.ScannerActivity
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Алгоритм перехода с мономодульности на многомодульность

Жизнь — суровая штука. И реальность такова, что работаем мы все с легаси. Если кто-то сейчас пилит новенький проект, где можно все сразу заправославить, то я тебе завидую, бро. Но у меня не так, и у того парня тоже не так =).

Как переводить ваше приложение на многомодульность? Я слышал в основном про два варианта.
Первый. Разбиение приложения на модули здесь и сейчас. Правда, у вас проект может месяц-другой вообще не собираться =).
Второй. Стараться вытаскивать фичи постепенно. Но заодно тянутся еще всякие зависимости этих фичей. И тут начинается самое интересное. Код зависимостей может тянуть еще другой код, все это дело мигрирует в *common-модуль*, в *core-модуль* и обратно, и так по кругу. В итоге вытягивание одной фичи может повлечь работу с еще доброй половиной приложения. И снова в начале ваш проект не будет собираться приличный отрезок времени.

Я выступаю за постепенный перевод приложения на многомодульность, так как нам параллельно нужно еще пилить новые фичи. Ключевая идея в том, что **если вашему модулю нужно что-то из зависимостей, не стоит сразу этот код физически также перетаскивать в модули**. Давайте рассмотрим алгоритм выноса модуля на примере Сканера:

- Создать апи фичи, поместить его в новый api-модуль. То есть полностью создать модуль *:feature-scanner-api* со всеми интерфейсами.
- Создать *:feature-scanner-impl*. В этот модуль физически перенести весь код, относящийся к фиче. Все, от чего зависит ваша фича, студия сразу подсветит.
- Выявить внешние зависимости фичи. Создать соответствующие интерфейсы. Эти интерфейсы разбить на логические api-модули. То есть в нашем примере создать модули *:core-utils, :core-network-api, :core-db-api, :feature-purchase-api* с соответствующими интерфейсами.
  Советую все-таки сразу вкладываться в название и смысл модулей. Понятно, что с течением времени интерфейсы и модули могут немного перетасовываться, схлопываться и т. д., это нормально.
- Создать апи внешних зависимостей (*ScannerFeatureDependencies*). В зависимости *:feature-scanner-impl* прописать созданные недавно api-модули.
- Так как в *app* у нас находится все легаси, то вот что делаем. В *app* мы подключаем все созданные для фичи модули (api-модуль фичи, impl-модуль фичи, api-модули внешних зависимостей фичи).
  Супер важный момент. Далее в *app* создаем имплементации всех необходимых интерфейсов зависимостей фичи (Сканера в нашем примере). Данные имплементации будут скорее просто проксями от ваших апи зависимостей к текущей реализации этих зависимостей в проекте. При инициализации компонента фичи подставляете данные имплементации.
  Сложно словами, хотите пример? Так он уже есть! По сути что-то подобное уже есть в feature-scanner-example. Еще раз приведу его немного адаптированный код:

```
// создаем вот такую реализацию ScannerFeatureDependencies в app-модуле
public class ScannerFeatureDependenciesLegacy implements ScannerFeatureDependencie

    @Override
    public DbClientApi dbClient() {
        return new DbClientLegacy();
    }

    @Override
    public HttpClientApi httpClient() {
        // какое-то легаси
        // главное, что мы имплементируем наш апи
        return NetworkFabric.createHttpClientLegacy();
    }

    @Override
    public SomeUtils someUtils() {
        return new SomeUtils();
    }

    @Override
    public PurchaseInteractor purchaseInteractor() {
        return new PurchaseInteractorLegacy();
    }

}

// и где-нибудь инициализируем граф
ScannerFeatureComponent.initAndGet(
    new ScannerFeatureDependenciesLegacy()
);
```

  То есть основной посыл здесь такой. Пусть весь необходимый для фичи внешний код живет в *app*, как и жил. А сама фича уже будет с ним работать по-нормальному, через апи (имеется в виду апи зависимостей и api-модули). В дальнейшем имплементации будут постепенно переезжать в модули. Но зато мы избежим бесконечной игры с перетаскиванием из модуля в модуль необходимого внешнего для фичи кода. Мы сможем двигаться четкими итерациями!
- Profit

Вот такой нехитрый, но работающий алгоритм, позволяющий двигаться к вашей цели шаг за шагом.

Дополнительные советы

Насколько большими/мелкими должны быть фичи?
Все зависит от проекта и т.д. Но в начале перехода на многомодульность я советую дробить по крупным кускам. Далее уже при необходимости будете из этих модулей выделять еще модули. Но не мельчите. Не делайте вот это: один/несколько классов = один модуль.

Чистота app-модуля
При переходе на многомодульность *app* у нас будет довольно большим, и оттуда в том числе будут дергаться ваши выделенные фичи. Не исключено, что в ходе работ вам придется вносить правки в это легаси, что-то там допиливать, ну или у вас просто релиз, и вам не до распилов на модули. В этом случае вы хотите, чтобы *app*, а вместе с ним и все легаси, знали о выделенных фичах только через апи, никакого знания об имплементациях. Но ведь *app*, по сути, соединяет в себе *api-* и *impl-модули*, а потому *app* знает обо всех.
В этом случае вы можете создать специальный модуль *:adapter*, который как раз и будет соединительной точкой api и impl, а уже *app* тогда будет знать только об api. Думаю, идея понятна. Пример вы можете посмотреть в ветке clean_app. Добавлю, что с Moxy, а точнее MoxyReflector, есть некоторые проблемы при дроблении на модули, из-за которых пришлось создать еще один дополнительный модуль *:stub-moxy-java*. Легкая щепотка магии, куда уж без нее.
Единственная поправка. Это сработает только тогда, когда ваша фича и соответствующие зависимости уже вынесены физически в другие модуля. Если вы вынесли фичу, но зависимости живут еще в *app*, как в алгоритме выше, то такое не получится.

Послесловие

Статья получилась немаленькой. Но я надеюсь, что она действительно поможет вам в борьбе с мономодульностью, осознанием, как это должно быть, и как это подружить с DI.
Если вам интересно погрузиться в проблему со скоростью сборки, как все мерить, то я рекомендую доклады Дениса Неклюдова и Жени Суворова (Mobius 2018 Piter, видео публично пока не доступны).
Про Gradle. Разницу между api и implementation в gradle отлично показал Вова Тагаков. Хотите уменьшить бойлерплейт многомодульности, можно начать вот с этой статьи.
Буду рад комментариям, поправкам, а также лайкам! Всем чистого кода!

Теги: android, android development, dagger 2, java, architecture design, mobile development, modules, multimodality

Хабы: Блог компании «Лаборатория Касперского», Java, Разработка мобильных приложений, Разработка под Android

+23 228 18

Редакторский дайджест
Присылаем лучшие статьи раз в месяц
Электропочта

«Лаборатория Касперского»
Ловим вирусы, исследуем угрозы, спасаем мир
Хабр Карьера Facebook Twitter ВКонтакте Instagram Facebook ВКонтакте

68 Карма 0 Рейтинг
Евгений Мацюк @xoxol_89
Software engineer
Facebook Github

Комментарии 18

Scrobot 13 сен 2018 в 12:41

Долгожданный ресерч от на тему микро-сервисов в андроиде) Огромное спасибо за статью)
Однозначно плюсую, но у меня есть пару вопросов:
1. Как мульти-модульная архитектура ведет себя при тестировании? Нужно же, помимо обычных юнит-тестов в каждом модуле, писать интеграционные тесты на стыке 2,3,N модулей. Насколько сильно это может мешать или помогать?
2. CleanArch сегодня рекомендуют использовать практически в каждом проекте, вне зависимости от масштаба. Что скажешь по поводу много-модульности? Стоит ли ее юзать в энтерпрайзе или даже в небольшом аутсорсе от нее можно получить свои бенефиты?
3. А что насчет оверхеда? Размер .apk, путаница с повторным подключением библиотек? Насколько это может быть опасно и какие моменты надо продумать, прежде чем решить переводить свой проект на мульти-модульную архитектуру. Усложняет ли это разработку в команде или упрощает? Ведь придется наверное немного пересмотреть воркфлоу под такую архитектуру, имхо.
4. В данном контексте удачно вписывается Dagger. Что насчет других DI фреймворков? Toothpick, Koin, Kodein? Мне вот например очень сложным представляется тут возможность юзануть Koin...

Ну и под конец: Где-то на рэдите как-то прочитал постик, про то, что связь мульти-модульных архитектур можно реализовать по аналогии с бэкендом — посредством вызова удаленных процедур. Тогда каждый модуль является автономным, а его функции цепляются по RPC вызовам. Мне показалось это как-то оверхедно, но интересно. Хотелось бы услышать и ваше мнение)

Ответить

xoxol_89 30 сен 2018 в 12:46

По вопросам:

1. По вопросам тестов пока придерживаюсь следующей картины мира.
Есть юнит-тесты. Это тестирование конкретного класса. Может быть тестирование совокупности классов (пример — тестирование интерактора, который внутри у себя дергает еще некоторые вспомогательные классы). Правда, в этом случае можно относить такие тесты к интеграционным, но все-таки тут тестируем конкретный юнит, который представлял из себя только один класс, но в процессе написания и дополнений распался на вспомогательные классы (SOLID и вот это все).
Все это на JUnit4/JUnit5/Spek.
Далее идут интеграционные и e2e тесты. Причем e2e тесты легко превращаются в интеграционные подменой нижнего слоя (например, Репозитория, или модуля, отвечающего за работу с сетью). Все это на Espresso + Kakao. Вот тут как раз можно спокойно тестировать конкретный фиче-модуль.
Тестирование заимодействия же модулей больше уходит в сторону e2e тестов, и тут с модулями у вас или нет — особой разницы нет.

2. По поводу повсеместного использования многомодульности. Как я уже говорил, многомодульность отлично решает вопрос корректного разбиения по фичам (разбиение по слоям делает CleanArch). Вы сразу же проектируете и пишите исходя из того, что каждая фича должна иметь конкретные контракты и их взаимодействие также должно подчиняться конкретным правилам. Прям архитектурный экстаз =)
Да, может быть определенный оверхед, но про Клин также говорили.
Может быть только в аутсорсе я бы упростил немного. Все в один модуль, а там разбиение по пакетам вместо разбиения по модулям.

3. Оверхед может возникнуть, например, с нативом. Каждый модуль будет формировать свои сошки, и это может нехило дать прирост в размере. Выход тут прост, весь натив у вас должен быть в одном модуле.
По поводу воркфлоу и командной работы. Мне кажется, что упростит работу и позволит вам, например, работать по Trunk-based development. Ведь обычный git-flow в команде уже 5+ разработчиков может вылиться в +100500 веток. А тут у вас вся разработка будет вестись в модулях и при необходимости вы сможете отключать не просто фиче-флаги, но весь ненужный код с ресурсами.

4. Другие DI фреймворки я не пробовал. Хватает Dagger 2 пока что =)

Про RPC-вызовы и вот это все. Ну в целом Гугл уже продвигает тему динамических фич, так что мы постепенно становимся еще ближе к бекенду =)

Ответить

nekdenis 17 сен 2018 в 07:28

Как адепт подобного подхода — одобряю! Спасибо, Евгений, что опубликовал столь информативный труд!

Ответить

Fi5t 17 сен 2018 в 13:15

Подход хороший, но я вижу в нем одну проблему, красивого решения которой я пока не придумал: шаринг моделей между фичами. Если какая-то модель нужна 2м фичам из 50-ти, то придется ее выносить в какую-то core сущность. И тут вилка или у нас будут плодиться такие core-сущности или распухать одна. Все становиться гораздо хуже, когда в проекте появляется room и тут уже вообще не очень понятно кто про кого должен знать учитывая специфику этой ORM. Если кто-то уже придумал как это все красиво разрулить, то буду рад вашим комментариям.

Ответить

xoxol_89 30 сен 2018 в 13:18

Ну я бы зашел с такой стороны.
Одна фича может знать о другой, но не наоборот. То есть нужно исключать двусвязные зависимости, только однонаправленные. Соответственно, модели данных для шаринга логично расположить в апи фичи, о которой знают. То есть фича А знает о фиче Б, то тогда модели данных для шаринга располагаются в апи фичи Б.

Ответить

alexeenkoff 13 мая 2019 в 12:21

Я правильно понимаю, что при таком подходе сильно раздуваются интерфейсы модулей core-db-api и core-network-api (и их имплементации соответственно)? Я в поисках решения для реального переиспользования модулей в других проектах. И при таком подходе предвижу уйму методов в `HttpClientApi` и в `DbClientApi`. Которые, к тому-же, не окажутся пригодными для переиспользования в условном другом проекте.

Ответить

xoxol_89 15 мая 2019 в 22:09

Не совсем понял, почему раздуваются?
Данные модули просто предоставляют клиентов для работы с бд, сетью. А модели, конкретные запросы и вот это все — это дело фичи.

Ответить

FirsofMaxim 27 ноя 2019 в 14:38

А что делать если скажем один и тот же репозиторий нужен для 2х разных фич? Выносить его как core модуль? Или feature?

Ответить

FirsofMaxim 13 сен 2019 в 11:23

Спасибо за статью, у меня есть сервис по обнаружению BlueTooth устройств в приложении, этим сервисом пользуются несколько разных экранов со своим workflow, я правильно понимаю стоит сделать features для:
1. сервиса BT устройств
2. сценарий #1
3. сценарий #2
?

Ответить

xoxol_89 26 сен 2019 в 07:41

похоже на правду

Ответить

FirsofMaxim 27 ноя 2019 в 09:03

Вопрос еще такой, а в каком месте например нужно инициализировать базу данных для приложения (например Room, там где все миграции, конвертертеры и нужен системный контекст для создания билдера базы данных)? Понятно что где-то в *core-db-impl*.

Ответить

Neikist 22 дек 2020 в 00:42

> Добавлю, что с Moxy, а точнее MoxyReflector, есть некоторые проблемы при дроблении на модули, из-за которых пришлось создать еще один дополнительный модуль :stub-moxy-java. Легкая щепотка магии, куда уж без нее.

А что за проблемы и что за магия? А то вот сегодня столкнулся с тем что презентер и вью в api описаны, реализация в impl, и в итоге дебаг версия нормально собирается, а релизная сборка падает.

Ответить

Georrg 11 сен 2021 в 17:31

Три года назад, в это самое время я ковырял outsource-ный проект на Cordove, паралельно пытаясь чего там накодить на котлине для внутренних нужд компании, и вот я здесь) Очень скоро наш проект нагонит все последние тренды - от Кордовы слава богу отказались как два года. Спасибо вам за статью. Интересно было бы узнать ваше внемние спустя три года? Чтобы сделали теперь иначе, лучше?

Ответить

FirsofMaxim 11 сен 2021 в 19:54

Ну я перехожу на другую реализацию: https://habr.com/ru/company/yandex/blog/419295/

Ответить

xoxol_89 15 сен 2021 в 20:14

Дык подход, описанный здесь, и то, что рассказывал Володя, мега коррелируют друг с другом =) Разве не так?

Ответить

FirsofMaxim 15 сен 2021 в 22:42

Так-то да, но дьявол в деталях.

Ответить

xoxol_89 16 сен 2021 в 14:56

Расскажи, интересно же =)

Ответить

FirsofMaxim 17 сен 2021 в 09:33

Провел ревью своего старого проекта и схватил facePalm (нашел ошибку в формировании dagger-компонента), ты прав концептуально одинаковый подход.

Ответить

Только полноправные пользователи могут оставлять комментарии. Войдите, пожалуйста.

Публикации
ЛУЧШИЕ ЗА СУТКИ ПОХОЖИЕ

divolko3 10 часов назад
Чернила дороже золота и блокировки из-за неоригинальных картриджей: у HP теперь проблемы с продажей принтеров
3 мин 14K
+25 12 59 +59

Seleditor 20 часов назад
Мини-ПК для «тяжелых» и не очень задач: 5 моделей начала весны 2024 года
3 мин 9.6K
+24 24 4 +4

Antxak 4 часа назад
Фишинг «фичи» Телеграма
Простой 4 мин 2.4K
Мнение
+23 9 5 +5

gal_may 8 часов назад
О найме. Взгляд оттуда
Простой 20 мин 8.2K
Из песочницы Recovery Mode
+21 33 27 +27

Bright_Translate 8 часов назад
Иммутабельность в механизме Durable Execution: проблемы и решение

Средний 7 мин 493

Кейс Перевод

+20 12 0

IPOSharks 6 часов назад

Технологические компании, в которые нельзя было вкладываться – крупнейшие провалы 2023

Простой 10 мин 4.8K

Дайджест

+16 7 5 +5

GlobalSign_admin 3 часа назад

OpenWrt отмечает 20 лет выпуском собственного маршрутизатора

3 мин 3.5K

+14 20 2 +2

3draven вчера в 23:02

Личное облако на Proxmox

Средний 8 мин 10K

Туториал

+13 99 30 +30

Frazick 10 часов назад

Apple M1 — разбор с точки зрения аппаратной безопасности

Простой 11 мин 4.4K

Аналитика

+11 13 6 +6

GidraVydra 8 часов назад

Теория химического строения. Ликбез. Часть 1

Простой 6 мин 1.3K

+10 15 6 +6

Показать еще

ВАКАНСИИ КОМПАНИИ ««ЛАБОРАТОРИЯ КАСПЕРСКОГО»»

Development Team Lead
Лаборатория Касперского · Москва

Software Development Engineer in Test
Лаборатория Касперского · Можно удаленно

Sales support manager
Лаборатория Касперского · Можно удаленно

Technical Product Trainer
Лаборатория Касперского · Москва

Product Designer B2B
Лаборатория Касперского · Москва

Больше вакансий на Хабр Карьере

ИНФОРМАЦИЯ

Сайт	www.kaspersky.ru
Дата регистрации	9 августа 2008
Дата основания	21 июля 1997
Численность	1 001–5 000 человек
Местоположение	Россия

ССЫЛКИ

Сайт «Лаборатории Касперского»
www.kaspersky.ru

Блог Евгения Касперского
eugene.kaspersky.ru

Блог для экспертов
securelist.ru

Блог о кибербезопасности
www.kaspersky.ru

Карьера в «Лаборатории»
vk.com

БЛОГ НА ХАБРЕ

27 фев в 00:08
Security Week 2409: PrintListener, кража отпечатков пальцев по стороннему каналу
1.3K 0

21 фев в 12:54
Приглашаем на онлайновый митап про карьеру техписа: наём, развитие, треки
416 1 +1

20 фев в 14:38
Может рухнуть при первой атаке. Почему компаниям важно прокачивать киберграмотность
9.7K 2 +2

19 фев в 22:20
Security Week 2408: уязвимости в Microsoft Exchange и Outlook
2.1K 1 +1

16 фев в 14:01
Трагические микрозаймы, украденные креды: какие Android-зловреды мы обнаружили в официальных маркетах
3.5K 8 +8

© 2006–2024, Habr